CS301: Silberschatz - OS Concepts (Ch. 8 - 10th Edition Revised)

Chapter 8: Deadlocks (10th Edition Revised)

8.1 System Model & Modern Multicore Architectures

In revised multicore architectures, resources are categorized with hardware affinity.

A process must request a resource before using it and must release it afterwards.

Deadlock occurs when every thread in a set is blocked waiting for an event that only another blocked thread in the same set can trigger.

CS301: Silberschatz - OS Concepts (Ch. 8 - 10th Edition Revised)

8.2 Deadlock Characterization: Updated Coffman Criteria

The 10th edition refines the classical 1971 Coffman definition for modern distributed locks:

1. Mutual Exclusion: At least one resource must be held in an exclusive lock mode.
2. Hold and Wait: A process holds at least one resource and requests additional locks.
3. No Preemption: Resources cannot be preempted; lock release is cooperative only.
   Revision note: Non-cooperative thread termination now introduces orphan mutex issues.
4. Circular Wait: A closed directed cycle of lock dependencies exists across executing threads.

CS301: Silberschatz - OS Concepts (Ch. 8 - 10th Edition Revised)

8.3 Clarified Starvation Metrics & Benchmarks

"Revised Definition: Deadlock is a static terminal state with 0% throughput across the blocked set."

"Starvation is a dynamic performance pathology where system throughput continues normally while specific thread

Statistical benchmarks show starvation occurs 14x more frequently than deadlock in cloud clusters due to non-uniform priority queuing and tail-latency scheduling anomalies.

CS301: Silberschatz - OS Concepts (Ch. 8 - 10th Edition Revised)

8.4 Banker’s Algorithm with Vector Extensions

Modern Banker’s Algorithm implementations employ SIMD vector instructions for fast safety checking. The safety algorithm complexity remains O(m * n^2), where m is resource types and n is processes. Safety invariant: Allocation + Need <= Available vector.